

AN2DL - First Challenge Report - NeuroNet

Mariam Marzougui, Massimo Albino Sandretti, Sushrut Mokasdar

mariemmrzg, massimoasandretti, sushrutmokasdar

295389, 269948, 232209

1 Introduction

In this project, we address a *time series classification* problem: a supervised learning task based on multivariate time series data. The goal is to predict the level of pain experienced by each subject, with 3 possible classes: **no_pain**, **low_pain**, **high_pain**.

2 Problem Analysis

The provided dataset consists of multivariate time series describing the temporal evolution of sensor readings for multiple subjects (identified by **sample_index**) and other non time-dependent information, such as the number of limbs/eyes. Each sequence consists of 160 time steps. The goal is to predict a pain level label for each subject.

Dataset Structure

Each record corresponds to a specific subject and time step and includes four main groups of features:

- **sample_index**: subject identifier
- **time**: timestep
- **Pain survey features** (**pain_survey_1--4**): rule-based pain indicators
- **Anatomical descriptors** (**n_legs**, **n_hands**, **n_eyes**): subject characteristics
- **Joint angles** (**joint_00--30**): continuous motion sensor readings

Main Challenges

- **Temporal dependency**: the model must recognize patterns distributed across time rather than from isolated measurements.
- **Heterogeneous feature types**: categorical, rule-based, and continuous time-series signals must be integrated into a single model.
- **Imbalanced classes**: preliminary label inspection shows uneven class distribution.
- **Absence of a validation split**: no official validation strategy is provided.

Initial Assumptions

Based on an exploratory analysis of the data, we assume that:

- anatomical descriptors (**n_legs**, **n_hands**, **n_eyes**) are provided as textual categories. In the dataset we observed that each attribute can take the value **"two"** or other strings representing any reduced count (e.g. **"one+peg_leg"**). For modelling simplicity, we therefore assume a binary interpretation: **"two"** indicates normal anatomy, while any other string is mapped to a single "non-two" category.

These observations guided the design of the pre-processing pipeline, the windowing strategy, and the choice of model architecture adopted in the subsequent sections.

3 Method

Our approach is structured a pipeline composed of four main stages: data preprocessing, subject-level splitting, sequence construction, and a recurrent neural network for final classification.

Data Preprocessing

We first group records by `sample_index`, so that each subject is represented as a time series based on the `time` field. Static anatomical descriptors (`n_legs`, `n_hands`, `n_eyes`) are provided in textual form. Based on dataset inspection, we assume each descriptor can take the value "two" or any other string representing an abnormal configuration (e.g., "one", "one+peg_leg"). For modelling purposes, we map:

$$\text{"two"} \mapsto 1, \quad \text{any other value} \mapsto 0,$$

yielding a compact binary representation.

All features are scaled using min-max normalization, computed on the training split.

Validation Strategy

Since the dataset does not include a predefined validation split, we used *stratified sampling* to get a more stable approximation of the actual score. Using 75/25 instead of 80/20 split also improved the actual score.

Windowing and Sequence Construction

To feed sequences into the recurrent model, we segment each subject's full time series into overlapping windows of fixed length:

$$X^{(k)} = [x_t, x_{t+1}, \dots, x_{t+L-1}],$$

where L is the **window size** and the **stride** controls the overlap. Each window inherits the subject-level label, meaning that the model learns to infer global pain status from partial temporal observations.

This strategy increases the number of training samples and allows the recurrent network to focus on local temporal patterns.

Model Architecture

We adopt a classifier based on a stack of bidirectional Gated Recurrent Units (GRUs). Given an

input window $X = [x_1, \dots, x_L] \in \mathbb{R}^{L \times d}$, where d is the feature dimension, the GRU updates the hidden state h_t at each time step t according to:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t]) \quad (1)$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t]) \quad (2)$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t]) \quad (3)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \quad (4)$$

Loss Function and Optimisation

We train the model using the cross-entropy loss:

$$\mathcal{L} = - \sum_{c=1}^C y_c \log(\hat{y}_c), \quad (5)$$

with $C = 3$ classes. Class weights are included to address label imbalance and to penalize misclassifications of rare classes. We tried both inverse frequency and square root of inverse frequency. The latter giving better results.

The network is optimized using the AdamW optimizer, with learning-rate scheduling and early stopping based on the validation weighted F1 score.

4 Experiments

The main experiment was conducted using the GRU model. We conducted a complementary experiment using a **BiLSTM** model to evaluate whether explicitly modeling long-term temporal dependencies could further improve classification performance, the same pipeline was used but we used a 80/20 split for LSTM. A hyperparameter search was also performed for the BiLSTM model, exploring combinations of parameters summarised in Table 2.

5 Results

The tables below show the results obtained. Although competitive, none of the BiLSTM configurations outperformed the strongest GRU model in our experiments.

6 Discussion

In this section, we analyze the performance of our model.

Table 1: Model configuration and performance.

Type	Layers	HSize	Bi	Dropout	L1/L2	LR	Epochs	Patience	Win/Stride	Acc.	Prec.	Recall	F1
GRU	3	64	Yes	0.35	0/1e-5	1e-3	150	20	20/4	0.9351	0.9338	0.9351	0.9343
GRU	3	128	Yes	0.2	0/1e-5	1e-3	150	20	20/4	0.9178	0.9168	0.9178	0.9165
LSTM	3	128	Yes	0.2	0/1e-5	1e-3	80	20	20/4	0.9100	0.9095	0.9100	0.9082

Table 2: Summary of BiLSTM hyperparameter tuning results (6 runs).

Window	Stride	HSize	Layers	Dropout	LR	Best Epoch	Val F1
20	10	128	2	0.3	5e-4	38	0.9002
20	10	256	2	0.3	1e-3	27	0.9001
20	10	128	2	0.3	1e-3	29	0.8847
20	10	256	2	0.3	5e-4	40	0.8986
40	10	128	2	0.3	1e-3	34	0.8813

Strengths

- **Temporal modeling:** The GRU network was able to learn from time dependent data in the multivariate time series, achieving a high F1 score on the validation and test sets.
- **Handling heterogeneous features:** By combining static subject information, joint angles, and pain survey features, the model successfully learned from multiple types of input data.
- **Regularization and early stopping:** Dropout and L2 regularization, combined with early stopping based on the F1 score, helped to mitigate overfitting.
- GRUs converged faster and achieved better F1 scores overall.
- LSTMs showed greater stability at lower learning rates.

Weaknesses

- **Overfitting:** Experimental results suggest that, despite the implemented regularization strategies, the model still tends to overfit.
- **Rare classes:** Despite class weighting, model training is affected by heavily underrepresented classes, potentially influencing the generalization of the model for these labels.

7 Conclusions

In this project, we addressed a multivariate time series classification task to predict pain levels from heterogeneous data. We designed a complete pipeline including preprocessing, subject-level validation, and a recurrent architecture based on GRU layers, which achieved strong performance on the validation set.

The main contributions of this work include:

- A tailored preprocessing and normalization pipeline for heterogeneous time-series data.
- A GRU-based classifier trained on overlapping temporal windows, effectively capturing local temporal patterns.
- Regularization and early stopping strategies that mitigated overfitting despite class imbalance.
- A complementary BiLSTM experiment and targeted hyperparameter tuning, confirming that GRUs remained the strongest architecture in our setup.

While the BiLSTM model and tuning search provided valuable insights, they did not surpass the performance of the best GRU configuration. Future improvements may involve exploring more advanced architectures (e.g., Transformers), refined tuning, or strategies addressing class imbalance such as data augmentation or focal loss.